



The University of Azad Jammu & Kashmir, Muzaffarabad

Department of Software Engineering

Database Systems

Open Ended Lab

Task Title: Car Rental Management System

Submitted to: Engr. Muhammad Awais

Student Name: Feeza Kulsoom

Roll No: 2024-SE-03

Reg Number: 2024-UMDB-004738

CAR RENTAL MANAGEMENT SYSTEM

Task 1: Database Design and Implementation

1.1 Entity Identification and Justification

Based on the CarGo Rentals scenario, the following four core entities are identified:

Customer

Stores information about customers who rent vehicles.

Important attributes:

- `customer_id`
- `name`
- `phone`
- `Email`

Vehicle

Stores information about vehicles available for rental.

Important attributes:

- `vehicle_id`
- `vehicle_number`
- `model`
- `daily_rate`
- `status`

Rental

Stores individual rental transactions between customers and vehicles.

Important attributes:

- rental_id
- customer_id
- vehicle_id
- rental_date
- return_date
- total_charge
- status

Payment

Stores payment information associated with rental transactions.

Important attributes:

- payment_id
- rental_id
- payment_date
- amount
- payment_status

This design separates customer, vehicle, rental, and payment information to reduce redundancy and prevent update, insertion, and deletion anomalies.

1.2 Relational Schema

Customer

`Customer(customer_id, name, phone, email)`

Vehicle

`Vehicle(vehicle_id, vehicle_number, model, daily_rate, status)`

Rental

```
Rental(rental_id, customer_id, vehicle_id, rental_date, return_date,  
total_charge, status)
```

Payment

```
Payment(payment_id, rental_id, payment_date, amount, payment_status)
```

1.3 Primary Keys

The following primary keys uniquely identify records:

- `Customer.customer_id`
- `Vehicle.vehicle_id`
- `Rental.rental_id`
- `Payment.payment_id`

1.4 Foreign Keys

The following foreign-key relationships maintain referential integrity:

- `Rental.customer_id` → `Customer.customer_id`
- `Rental.vehicle_id` → `Vehicle.vehicle_id`
- `Payment.rental_id` → `Rental.rental_id`

1.5 Integrity Constraints

The database uses the following constraints:

- `PRIMARY KEY` to uniquely identify records.
- `FOREIGN KEY` to maintain relationships between tables.
- `NOT NULL` for essential attributes.
- `UNIQUE` for customer phone numbers, customer email addresses, and vehicle registration numbers.
- `CHECK` constraints for positive daily rates, positive payment amounts, valid statuses, and valid rental dates.

- DEFAULT values for vehicle and rental statuses.

The database also maintains vehicle availability so that a vehicle with an active rental cannot be registered for another active rental.

1.6 Database and Table Implementation

The complete SQL implementation is provided in the accompanying SQL file:

`car_rental_script.sql`

The script contains:

- Database creation
- Table creation
- Primary and foreign keys
- Integrity constraints
- Sample data
- JOIN queries
- View creation
- Stored procedure
- Trigger
- Optimization indexes

Task 2: Normalization Analysis

2.1 Unnormalized Form (UNF)

The original rental information is stored in a single structure:

RentalID, CustomerName, CustomerPhone, VehicleNumber, VehicleModel, DailyRate, RentalDate, ReturnDate, PaymentAmount

Example:

RentalID	CustomerName	CustomerPhone	VehicleNumber	VehicleModel	DailyRate	RentalDate	ReturnDate	PaymentAmount
R001	Usman Khan	0300-1234567	ABC-123	Toyota Corolla	5000	2026-09-11	2026-09-14	15000

This structure stores information about different entities in one record. If the same customer rents several vehicles, the customer information must be repeated.

This can cause:

- Data redundancy
- Update anomalies
- Insertion anomalies
- Deletion anomalies

2.2 First Normal Form (1NF)

A relation is in First Normal Form when:

- Each attribute contains atomic values.
- There are no repeating groups.
- Each row represents one record.

The rental information can therefore be represented as:

```
Rental(RentalID, CustomerName, CustomerPhone, VehicleNumber, VehicleModel,  
DailyRate, RentalDate, ReturnDate, PaymentAmount)
```

Each field contains a single atomic value.

Problem remaining after 1NF

Although all values are atomic, customer and vehicle information is still repeated whenever the same customer or vehicle participates in multiple rentals.

2.3 Second Normal Form (2NF)

2NF is concerned with eliminating partial dependencies when a relation has a composite key. In the Car Rental system, RentalID uniquely identifies each rental transaction, so the final Rental relation does not require a composite primary key. However, the original combined structure contains attributes describing different real-world entities. CustomerName and CustomerPhone describe the customer, while VehicleNumber, VehicleModel, and DailyRate describe the vehicle. These attributes should therefore be separated into their respective relations.

The resulting relations are:

Customer: Customer(CustomerID, Name, Phone, Email)

Vehicle: Vehicle(VehicleID, VehicleNumber, Model, DailyRate, Status)

Rental: Rental(RentalID, CustomerID, VehicleID, RentalDate, ReturnDate, TotalCharge, Status)

This decomposition removes unnecessary repetition of customer and vehicle information from rental records and establishes clear dependencies on the appropriate primary keys.

.

2.4 Third Normal Form (3NF)

The final design is in Third Normal Form because each relation is already in 2NF and non-key attributes depend on the primary key of their own relation rather than on another non-key attribute. Customer attributes depend on CustomerID, vehicle attributes depend on VehicleID, rental attributes depend on RentalID, and payment attributes depend on PaymentID. Separating these relations prevents customer, vehicle, rental, and payment information from being unnecessarily repeated. Payment information is separated into its own relation:

Payment(PaymentID, RentalID, PaymentDate, Amount, PaymentStatus)

The final relations are:

- Customer
- Vehicle
- Rental
- Payment

Each non-key attribute describes the entity represented by its table's primary key.

Problems resolved by normalization

Update anomaly:

Customer or vehicle information can be updated in one place instead of multiple rental records.

Insertion anomaly:

A new customer or vehicle can be added without requiring a rental transaction.

Deletion anomaly:

Deleting a rental does not automatically remove the customer's or vehicle's basic information.

Redundancy:

Repeated customer and vehicle information is reduced by separating the entities into individual tables.

Task 3: JOIN Queries

Query 1 – Display Every Rental

Requirement

Display the customer name, vehicle number, vehicle model, rental date, and return date for every rental.

SQL Statement

```
use carrentaldb;
SELECT
    c.name AS CustomerName,
    v.vehicle_number AS VehicleNumber,
    v.model AS VehicleModel,
    r.rental_date AS RentalDate,
    r.return_date AS ReturnDate
FROM Rental r
INNER JOIN Customer c
    ON r.customer_id = c.customer_id
INNER JOIN Vehicle v
    ON r.vehicle_id = v.vehicle_id;
```

Explanation

INNER JOIN is appropriate because the query should display rental records together with their corresponding customer and vehicle information.

✓ Showing rows 0 - 2 (3 total, Query took 0.0012 seconds.)

```
SELECT c.name AS CustomerName, v.vehicle_number AS VehicleNumber, v.model AS VehicleModel, r.rental_date AS RentalDate, r.return_date AS ReturnDate FROM Rental r INNER JOIN Customer c ON r.customer_id = c.customer_id INNER JOIN Vehicle v ON r.vehicle_id = v.vehicle_id;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

CustomerName	VehicleNumber	VehicleModel	RentalDate	ReturnDate
Usman Khan	ABC-123	Toyota Corolla	2026-09-11	2026-09-14
Ayesha Malik	XYZ-789	Honda Civic	2026-09-01	2026-09-05
Usman Khan	LHR-456	Suzuki Alto	2026-08-20	2026-08-22

☐ Show all | Number of rows: 25 | Filter rows:

Query 2 – All Customers and Their Rented Vehicles

Requirement

Display all customers and the vehicles they have rented. Customers who have never rented a vehicle must also appear.

SQL Statement

```
use carrentaldb;
SELECT
    c.customer_id AS CustomerID,
    c.name AS CustomerName,
    v.vehicle_number AS VehicleNumber,
    v.model AS VehicleModel
FROM Customer c
LEFT JOIN Rental r
    ON c.customer_id = r.customer_id
LEFT JOIN Vehicle v
    ON r.vehicle_id = v.vehicle_id;
```

Explanation

LEFT JOIN is used so that every customer appears even when no matching rental exists. Customers without rentals will have NULL values for vehicle information.

✓ Showing rows 0 - 5 (6 total, Query took 0.0313 seconds.)

```
SELECT c.customer_id AS CustomerID, c.name AS CustomerName, v.vehicle_number AS VehicleNumber, v.model AS VehicleModel FROM Customer c
LEFT JOIN Rental r ON c.customer_id = r.customer_id LEFT JOIN Vehicle v ON r.vehicle_id = v.vehicle_id;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

CustomerID	CustomerName	VehicleNumber	VehicleModel
1	Usman Khan	ABC-123	Toyota Corolla
2	Ayesha Malik	XYZ-789	Honda Civic
1	Usman Khan	LHR-456	Suzuki Alto
3	Ali Hassan	NULL	NULL
4	Zainab Bibi	NULL	NULL
5	Hamza Ahmed	NULL	NULL

☐ Show all | Number of rows: 25 ▾ Filter rows:

Query 3 – All Vehicles and Current Rental Information

Requirement

Display all vehicles and their current rental information. Vehicles that are currently not rented must also appear.

SQL Statement

```
use carrentaldb;
SELECT
    v.vehicle_id AS VehicleID,
    v.vehicle_number AS VehicleNumber,
    v.model AS Model,
    v.status AS CurrentStatus,
    r.rental_id AS RentalID,
    r.rental_date AS RentalDate,
    r.return_date AS ReturnDate
FROM Vehicle v
LEFT JOIN Rental r
    ON v.vehicle_id = r.vehicle_id
    AND r.status = 'Active';
```

```
use carrentaldb;
SELECT
    v.vehicle_id AS VehicleID,
    v.vehicle_number AS VehicleNumber,
    v.model AS Model,
    v.status AS CurrentStatus,
    r.rental_id AS RentalID,
    r.rental_date AS RentalDate,
    r.return_date AS ReturnDate
FROM Vehicle v
LEFT JOIN Rental r
    ON v.vehicle_id = r.vehicle_id
    AND r.status = 'Active';
```

Explanation

A `LEFT JOIN` ensures that every vehicle is displayed. The condition `r.status = 'Active'` ensures that only current active rental information is associated with the vehicle. Vehicles without an active rental display `NULL` rental information.

✓ Showing rows 0 - 4 (5 total, Query took 0.0014 seconds.)

```
SELECT v.vehicle_id AS VehicleID, v.vehicle_number AS VehicleNumber, v.model AS Model, v.status AS CurrentStatus, r.rental_id AS RentalID, r.rental_date AS RentalDate, r.return_date AS ReturnDate FROM Vehicle v LEFT JOIN Rental r ON v.vehicle_id = r.vehicle_id AND r.status = 'Active';
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

VehicleID	VehicleNumber	Model	CurrentStatus	RentalID	RentalDate	ReturnDate
1	ABC-123	Toyota Corolla	Rented	1	2026-09-11	2026-09-14
2	XYZ-789	Honda Civic	Available	NULL	NULL	NULL
3	LHR-456	Suzuki Alto	Available	NULL	NULL	NULL
4	KHI-999	Kia Sportage	Maintenance	NULL	NULL	NULL
5	ISB-555	Toyota Yaris	Available	NULL	NULL	NULL

☐ Show all | Number of rows: 25 | Filter rows:

Query 4 – Total Rentals per Customer

Requirement

Display the total number of rentals made by each customer, including customers who have made no rentals.

SQL Statement

```
use carrentaldb;
SELECT
    c.customer_id AS CustomerID,
    c.name AS CustomerName,
    COUNT(r.rental_id) AS TotalRentals
FROM Customer c
LEFT JOIN Rental r
    ON c.customer_id = r.customer_id
GROUP BY c.customer_id, c.name;
```

Explanation

LEFT JOIN ensures that customers with no rentals are included. COUNT(r.rental_id) counts only actual rental records, so customers without rentals receive a count of zero.

✓ Showing rows 0 - 4 (5 total, Query took 0.0018 seconds.)

```
SELECT c.customer_id AS CustomerID, c.name AS CustomerName, COUNT(r.rental_id) AS TotalRentals FROM Customer c LEFT JOIN Rental r ON c.customer_id = r.customer_id GROUP BY c.customer_id, c.name;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

CustomerID	CustomerName	TotalRentals
1	Usman Khan	2
2	Ayesha Malik	1
3	Ali Hassan	0
4	Zainab Bibi	0
5	Hamza Ahmed	0

☐ Show all | Number of rows: 25 | Filter rows:

Task 4: Database View

4.1 View Creation

A consolidated rental report is created to combine customer, vehicle, rental, and payment information.

SQL Statement

```
CREATE VIEW vw_ConsolidatedRentalReport AS
SELECT
    r.rental_id AS RentalID,
    c.name AS CustomerName,
    c.phone AS CustomerPhone,
    v.vehicle_number AS VehicleNumber,
    v.model AS VehicleModel,
    r.rental_date AS RentalDate,
    r.return_date AS ReturnDate,
    r.total_charge AS TotalCharge,
    p.payment_status AS PaymentStatus
FROM Rental r
INNER JOIN Customer c
    ON r.customer_id = c.customer_id
INNER JOIN Vehicle v
```

```

ON r.vehicle_id = v.vehicle_id
LEFT JOIN Payment p
ON r.rental_id = p.rental_id;

```

4.2 View Execution

```

use carrentaldb;
SELECT * FROM vw_ConsolidatedRentalReport;

```

4.3 Utility of the View

The view combines information from the Customer, Vehicle, Rental, and Payment tables into a single reporting structure. It simplifies repeated JOIN operations and provides management with a convenient way to review rental transactions, customer information, vehicle details, charges, and payment status.

✓ Showing rows 0 - 2 (3 total, Query took 0.0010 seconds.)

SELECT * FROM vw_ConsolidatedRentalReport;

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

		RentalID	CustomerName	CustomerPhone	VehicleNumber	VehicleModel	RentalDate	ReturnDate	TotalCharge
☐	Edit Copy Delete	1	Usman Khan	0300-1234567	ABC-123	Toyota Corolla	2026-09-11	2026-09-14	15000.00
☐	Edit Copy Delete	2	Ayesha Malik	0311-9876543	XYZ-789	Honda Civic	2026-09-01	2026-09-05	24000.00
☐	Edit Copy Delete	3	Usman Khan	0300-1234567	LHR-456	Suzuki Alto	2026-08-20	2026-08-22	6000.00

↑ ☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows:

PaymentStatus

Paid

Paid

Paid

Task 5: Stored Procedure

5.1 Purpose

The stored procedure registers a new rental, checks vehicle availability, calculates the rental charge according to the rental period and daily rate, inserts the rental record, and updates the vehicle status.

5.2 Stored Procedure

```
use carrentaldb
DELIMITER //

CREATE PROCEDURE sp_RegisterNewRental(
    IN p_customer_id INT,
    IN p_vehicle_id INT,
    IN p_rental_date DATE,
    IN p_return_date DATE
)
BEGIN
    DECLARE v_daily_rate DECIMAL(10,2);
    DECLARE v_days INT;
    DECLARE v_total_charge DECIMAL(10,2);
    DECLARE v_vehicle_status VARCHAR(20);
    DECLARE v_customer_count INT;

    SELECT COUNT(*)
    INTO v_customer_count
    FROM Customer
    WHERE customer_id = p_customer_id;

    IF v_customer_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Customer does not exist.';
    END IF;

    SELECT COUNT(*)
    INTO v_vehicle_count
    FROM Vehicle
    WHERE vehicle_id = p_vehicle_id;
```

```

IF v_vehicle_count = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error: Vehicle does not exist.';
END IF;

SELECT status, daily_rate
INTO v_vehicle_status, v_daily_rate
FROM Vehicle
WHERE vehicle_id = p_vehicle_id;

IF v_vehicle_status <> 'Available' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error: Vehicle is not available for rental.';
END IF;

IF p_return_date < p_rental_date THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error: Return date cannot be before rental date.';
END IF;

SET v_days = DATEDIFF(p_return_date, p_rental_date);

IF v_days <= 0 THEN
    SET v_days = 1;
END IF;

SET v_total_charge = v_days * v_daily_rate;

INSERT INTO Rental(
    customer_id,
    vehicle_id,
    rental_date,
    return_date,
    total_charge,
    status
)
VALUES(
    p_customer_id,
    p_vehicle_id,
    p_rental_date,
    p_return_date,
    v_total_charge,
    'Active'
);

UPDATE Vehicle

    SET status = 'Rented'
    WHERE vehicle_id = p_vehicle_id;

END //

DELIMITER ;

```

5.3 Procedure Execution

Use Vehicle 3 because it is initially Available.

```
use carrentaldb;
CALL sp_RegisterNewRental(
    3,
    3,
    '2026-09-22',
    '2026-09-25'
);
```

Then verify the inserted rental:

```
SELECT *
FROM Rental
WHERE customer_id = 3
AND vehicle_id = 3;
```

Verify that the vehicle status changed:

```
SELECT *
FROM Vehicle
WHERE vehicle_id = 3;
```

The vehicle should now have status:

Rented

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)

```
use carrentaldb;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0491 seconds.)

```
CALL sp_RegisterNewRental( 3, 3, '2026-09-22', '2026-09-25' );
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

✓ Showing rows 0 - 0 (1 total, Query took 0.0007 seconds.)

```
SELECT * FROM Rental WHERE customer_id = 3 AND vehicle_id = 3;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	rental_id	customer_id	vehicle_id	rental_date	return_date	total_charge	status
☐ Edit Copy Delete	4	3	3	2026-09-22	2026-09-25	9000.00	Active

↑ ☐ Check all With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table

✓ Showing rows 0 - 0 (1 total, Query took 0.0005 seconds.)

```
SELECT * FROM Vehicle WHERE vehicle_id = 3;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	vehicle_id	vehicle_number	model	daily_rate	status
☐ Edit Copy Delete	3	LHR-456	Suzuki Alto	3000.00	Rented

↑ ☐ Check all With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Task 6: Trigger

6.1 Purpose

The trigger automatically changes a vehicle's status from `Rented` to `Available` when an active rental is completed.

6.2 Trigger Code

```
use carrentaldb;
DELIMITER //

CREATE TRIGGER trg_AfterRentalComplete
AFTER UPDATE ON Rental
FOR EACH ROW
BEGIN
    IF NEW.status = 'Completed'
        AND OLD.status = 'Active' THEN

        UPDATE Vehicle
        SET status = 'Available'
        WHERE vehicle_id = NEW.vehicle_id;

    END IF;

END //
```

```
DELIMITER ;
```

6.3 Trigger Verification

First, update an active rental:

```
use carrentaldb;
UPDATE Rental
SET status = 'Completed'
WHERE rental_id = 1;
```

Then verify the corresponding vehicle:

```
SELECT *
FROM Vehicle
WHERE vehicle_id = 1;
```

The vehicle status should change from:

Rented

to:

Available

✓ Showing rows 0 - 0 (1 total, Query took 0.0005 seconds.)

```
SELECT * FROM Vehicle WHERE vehicle_id = 1;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

	vehicle_id	vehicle_number	model	daily_rate	status
☐ Edit Copy Delete	1	ABC-123	Toyota Corolla	5000.00	Available

[↶](#) [↷](#) ☐ Check all *With selected:* [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows:

Task 7: Optimization Analysis

7.1 Identified Inefficiency

As the number of rental records increases, searching the Rental table by `customer_id` can require the database to examine a large number of rows if an appropriate index is not available.

For example, the following query is frequently useful when retrieving a customer's rental history:

```
SELECT *
FROM Rental
WHERE customer_id = 1;
```

Without an index on `customer_id`, the database may need to perform a full table scan.

7.2 Proposed Improvement

An index can be created on `Rental.customer_id`:

```
use carrentaldb;
CREATE INDEX idx_rental_customer
ON Rental(customer_id);
```

The index provides the database with an additional access path for locating rental records by `customer_id`, which can reduce the amount of data that must be examined as the Rental table grows.

7.3 Query Execution Plan

The execution plan can be inspected using:

```
use carrentaldb;
EXPLAIN
SELECT *
FROM Rental
WHERE customer_id = 1;
```

The index can also be checked using:

```
use carrentaldb;
SHOW INDEX FROM Rental;
```

Performance Analysis

The index does not change the logical result of the query. Instead, it provides a more efficient access path for searches based on `customer_id`, particularly when the Rental table contains a large number of records.

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
use carrentaldb;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.1287 seconds.)

```
CREATE INDEX idx_rental_customer ON Rental(customer_id);
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

```
use carrentaldb;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

Your SQL query has been executed successfully.

```
EXPLAIN SELECT * FROM Rental WHERE customer_id = 1;
```

[[Edit inline](#)] [[Edit](#)] [[Skip Explain SQL](#)] [[Create PHP code](#)]

Extra options

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	Rental	ref	idx_rental_customer	idx_rental_customer	4	const	2	

Your SQL query has been executed successfully.

```
SHOW INDEX FROM Rental;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)] [[Refresh](#)]

Extra options

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
rental	0	PRIMARY	1	rental_id	A	3	NULL	NULL		BTREE		
rental	1	fk_rental_vehicle	1	vehicle_id	A	3	NULL	NULL		BTREE		
rental	1	idx_rental_customer	1	customer_id	A	3	NULL	NULL		BTREE		

Comment	Index_comment
---------	---------------

Conclusion

The CarGo Rentals database provides a normalized relational solution for managing customers, vehicles, rental transactions, and payments. Primary keys, foreign keys, and integrity constraints maintain data consistency, while JOIN queries and the consolidated view support reporting requirements.

The stored procedure automates rental registration and charge calculation, while the trigger automatically updates vehicle availability when a rental is completed. Finally, indexing the frequently searched `customer_id` column improves the efficiency of customer rental-history queries as the database grows.